Object Oriented Programming Exam

The exam has five exercises:

- You must write Java code to solve the exercises.
- The exercises are chained. Each exercise builds partially on the previous ones.
- Do not create duplicated classes. Extend or modify the same class structure as needed.
- Some requirements are intentionally open. In these cases, you may choose a reasonable design, but you must explain and justify your choice using comments in the code.
- Submit a single, global solution covering all exercises.

Instructions:

- Students are allowed to use notes and course material (physical or electronic).
- Students are **not allowed** to query the web, use generative AI tools, or automatic code generation.
- Any IDE may be used (e.g., VS Code, IntelliJ), provided that no AI assistance is enabled.

Submission:

- Place all Java classes into a **zip file**, and submit it via Digital Exam.

Exercise 1

A smart parking system needs to manage vehicles and their parking positions inside a parking garage.

Part (a)

Create a Java class `ParkingSpot` that stores the `level` (String) and `spotNumber` (int) of a parking spot. Once initialized, these attributes **cannot be modified**.

Write the complete code for this class, including:

- Fields
- Constructor
- Getters
- A meaningful `toString` method

Part (b)

Create a Java class `Vehicle` to represent a vehicle in the parking garage.

Each `Vehicle` object must contain:

- A `String licensePlate` (unique identifier). This value must be initialized at creation time and **cannot be modified**.
- A `String brand`. This value must be initialized and **cannot be modified**.

Write the full class implementation, including:

- Constructor
- Getters and setters (where allowed)
- An overridden `toString` method including all attributes

Exercise 2

The parking system must support different types of vehicles.

Requirements

1. **Class** `ElectricCar`:
    - Inherits from `Vehicle`.
    - Contains an additional attribute:
        - `int batteryCapacity` (in kWh).
    - The `batteryCapacity` value must be initialized when the object is created and **cannot** be modified later.
2. **Class** `Motorcycle`:
    - Inherits from `Vehicle`.
    - Contains an additional attribute:
        - `boolean hasSidecar`.
    - The `hasSidecar` value must be initialized when the object is created and **cannot** be modified later.
3. Both `ElectricCar` and `Motorcycle` must override `toString` properly.

Exercise 3

The parking garage requires a centralized system to manage all parking spots and the vehicles parked at them. The garage is organized into **zones** (e.g., `"EV"`, `"General"`, `"Compact"`), and it has a maximum capacity.

Requirements

1. **Modify the existing** `ParkingSpot`:
   - Add a new attribute `String zone`. The `zone` must be initialized when the `ParkingSpot` object is created and **cannot** be modified later.
   - Two parking spots are considered equal if they have the same `level` and `spotNumber`.
2. **Class** `ParkingGarage`:The garage must provide a constructor:

   `ParkingGarage(int maxVehicles)`

   which creates suitable collections and enough `ParkingSpot` to park a vehicle. A `ParkingSpot` can only park one vehicle. This class also keeps track of all the parking spots available and the vehicles parked in the garage.
3. **Zone rule**: A vehicle can be parked only in a spot whose zone is compatible with the vehicle type:
   - `ElectricCar` can **only** be parked in a spot with zone `"EV"`.
   - `Motorcycle` can be parked in **any** zone.
4. **Vehicle management methods**:
   - `void parkVehicle(Vehicle vehicle)`:
     - Adds the given vehicle to the garage .A vehicle is parked in exactly one parking spot chosen from available spots.
     - If a vehicle with the same `licensePlate` already exists, throw `DuplicateVehicleException`.
     - If the garage already contains `maxVehicles` vehicles, calling `parkVehicle` must throw a custom `GarageFullException`.
     - If the zone rule is violated, throw `IncompatibleZoneException`.
   - `void removeVehicle(Vehicle vehicle)`:

- Removes the given vehicle from the garage if it exists.
- If the vehicle is not present, the method must do nothing.

- `ParkingSpot getParkingSpot(String licensePlate)`:
  - Returns `ParkingSpot` where the vehicle is parked.
  - If the vehicle is not present, the method must throw `NonPresentVehicleExceptio`

- `Collection<Vehicle> findVehiclesInZone(String zone)`:
  - Returns all vehicles parked in any spot whose zone matches the given `zone`.
  - If none are found, return an empty collection.

5. **Create Unit Tests** to verify the correct behavior of all the *Vehicle management methods* described above.

Exercise 4

The parking garage wants to compute parking fees using different pricing policies. Implement the **Strategy Design Pattern** to support interchangeable pricing strategies.

Requirements

1. Create an interface `PricingStrategy` with the method:

   ```
   double computeFee(Vehicle vehicle, int minutesParked, ParkingSpot spot);
   ```

2. Implement at least two concrete pricing strategies:

   - `FlatRatePricing` – Charges a fixed amount per started hour (rounded up).
   - `ZoneBasedPricing` – Charges different amounts per started hour depending on the zone (e.g., `"EV"` vs `"General"`).

3. Modify `ParkingGarage` so that it can be configured with a `PricingStrategy` and use it to calculate fees.

   Add a method to `ParkingGarage`:

   ```
   double calculateFee(String licensePlate, int minutesParked);
   ```

   - The method must use the configured `PricingStrategy`.

Exercise 5

The parking garage wants to automatically assign a parking spot to vehicles according to different policies. Implement the **Strategy Design Pattern** to support interchangeable spot allocation strategies.

Requirements

1. Create an interface `SpotAllocationStrategy`.
2. Implement at least two concrete allocation strategies:
   - `FirstAvailableStrategy` – Based on first available spot principle.
   - `CompactFirstStrategy` – Based on first available parking spot principle but in a "Compact" zone. If not possible (e.g., because "Compact" zone is fully occupied) changes to `FirstAvailableStrategy`.
3. Modify `ParkingGarage` so that it can be configured with a `SpotAllocationStrategy` and use it to automatically park vehicles.
4. Update the method `void parkVehicleAuto(Vehicle vehicle)` to select a spot using the configured `SpotAllocationStrategy` and park the vehicle using the existing parking logic and constraints from Exercise 3.